From Graphical Manuals to Robot Execution: Task and Skill-Level Behavior Trees for Assembly

Praveen Kumar Menaka Sekar[†1], Charles A. Meehan[†1], Ion Matei[2], Hon Yung Wong[2], and John S. Baras[3]

[1]Department of Mechanical Engineering, University of Maryland
[2]Fujitsu Research of America
[3]Department of Electrical Engineering, University of Maryland
[†]Equal Contribution

Abstract: Robots that assemble products from instructions written for humans face two errors that often remain unnoticed until a later assembly step fails: the vision-language model (VLM) that reads the instructions can misread a step, and the robot can execute a step imperfectly. We study where such a robot should inspect its work and when it should query a human operator. Experienced workers ask a supervisor when a drawing does not show which part goes where, check connectors before the next part covers them, and look closely before a tight insertion. We encode these behaviors in a hierarchical behavior tree (BT). Before execution, an off-the-shelf VLM answers one yes/no question per part and manual step, and a decoder combines the answers into an assembly plan with an error probability for each step. A task-level planner, a dynamic program, then synthesizes the task-level BT, which places inspections and operator queries at minimum expected cost under a bound on the expected number of undetected failures. Each assembly action of this BT expands into a skill-level subtree whose skills execute only inside success regions measured in simulation. On 21 products with an 8B VLM, the decoder recovers 42% of the annotated assembly operations, against 21% for a single plan query, and its mean error probability matches the fraction of operations that it decodes incorrectly. Querying the operator about uncertain steps reduces the expected number of undetected failures over the 21 products from 20.5 to 4.0. At the same risk, the synthesized BT costs 14–21% less than inspecting only the finished product or every operation, and up to 0.7% less than simple query rules that use the same error probabilities. In simulation, a Franka Panda completes an IKEA chair frame in 35 of 39 runs, with no undetected failure. Project page: https://praveen1098.github.io/IKEA-Assembly-Planning/.

Keywords: Behavior trees, robotic assembly, instruction following, vision-language models, planning under uncertainty

1 Introduction

Industrial assembly relies on instructions written for humans, such as work instructions, assembly drawings, and product manuals. Vision-language models (VLMs) can now extract assembly plans from such documents [1, 2], and language models can convert instructions into behavior trees (BTs) [3, 4]. Two errors remain. A misread step makes the robot attach the wrong part, and an imperfect execution can leave a connector unseated. Both errors tend to go unnoticed until a later part does not fit, which then requires disassembly. Current pipelines either execute the extracted plan as if it were correct [1, 2] or react only to the failures that they detect during execution [5, 6, 7]. Neither assumption holds: a single plan query with an 8B VLM recovers only 21% of the annotated

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

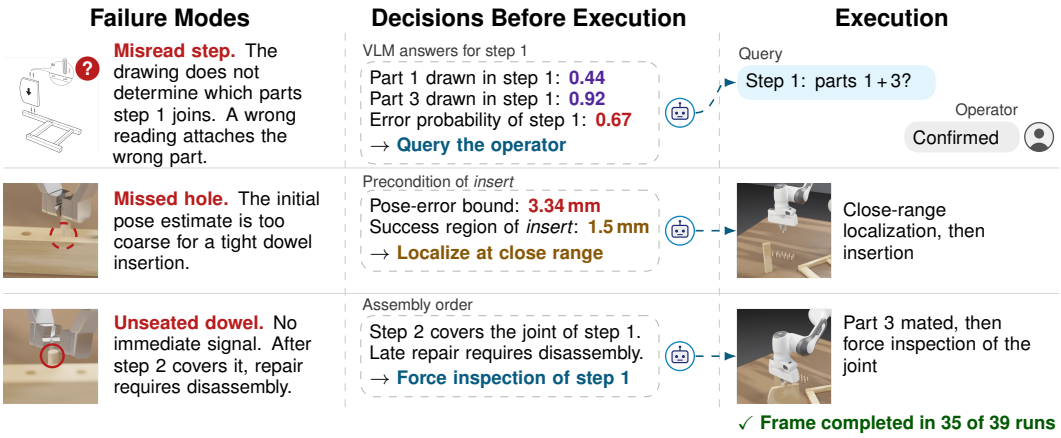


Figure 1: **Overview on the IKEA SUNDVIK chair frame.** Each row pairs a failure mode (left) with the decision computed before execution (middle) and the resulting action during execution (right). Inspecting and querying at every step is costly, so the task-level planner selects where to do each.

operations of our benchmark (Sec. 7), and VLMs that judge task success reach a balanced accuracy below 0.60 on contact-rich assembly [8]. Inspecting every step, in turn, adds time to every operation. This paper studies where a robot should inspect its work and when it should query an operator. We ground this problem in the assembly of IKEA furniture from its manuals.

Experienced assembly workers ask a supervisor when an instruction is ambiguous, check connectors before the next component covers them, and look closely before a tight insertion. We encode these behaviors in a hierarchical BT (Fig. 1). The VLM has two roles. Before execution, it answers one yes/no question per part and step, and a decoder combines the answers into a plan with an error probability per step. During execution, the VLM compares a camera image of the current assembly with the manual drawing of that step. This visual inspection complements a force inspection, which pushes the placed part along its insertion axis. The *task-level planner*, a dynamic program, synthesizes the task-level BT, which decides which operations to inspect and which steps to query. Each assembly action expands into a skill-level subtree whose skills execute only inside their success regions: the pose-estimate errors at which they succeed with probability at least 0.8 in simulation (Fig. 2).

The task-level planner never sees images or contact dynamics. It plans over an abstraction, the probabilistic task model (the assembly tree, an error probability per step, and failure and detection rates), and outputs a second abstraction, the BT, which the robot executes and a person reads. Two quantities are learned from data: the error probability of each step, from the VLM's answers, and the success region of each skill, from simulated executions. Sec. 7 tests when per-step error probabilities beat a single constant.

Inspection allocation and execution monitoring assume a correct plan, and methods that query for help assume correct execution (Sec. 2). Our contributions are: (1) a formulation of inspection and query placement on the assembly tree as one optimization against both misread steps and execution errors; (2) a structured decoder that assigns every step an error probability, and a dynamic program that returns the cheapest BT in a template family, with cost and risk exact under the task model; and (3) skill preconditions defined as the largest pose-estimate error that a skill tolerates in simulation, with a check before execution that close-range localization brings the error inside them.

2 Related Work

Assembly from instructions and BT synthesis. Manual2Skill [1] extracts an assembly graph from manual pages and assembles real furniture, and Manual2Skill++ [2], whose benchmark we use, adds connectors. Other systems build plans or BTs from demonstrations [5], CAD models [9], text

manuals [6], or language instructions [3, 4], without modeling that the plan or its execution may be wrong. POMDP solvers [10, 11] and RAO* [12] compute adaptive policies. We optimize over BT templates and measure what this restriction costs.

Inspection, asking for help, and skill preconditions. Inspection allocation places imperfect inspections by dynamic programming [13, 14]. Extensions cover branched assembly with rework [15], defects that are harder to inspect once covered [16], and monitoring of plan preconditions [17]. All of them assume a correct plan. KnowNo [18] and its successors [19, 20] query a human when a prediction set is ambiguous but assume that execution succeeds, and REFLECT [21] explains execution failures after they occur. Learned skill preconditions [22, 23] and B-CTMP [24] describe the start states from which a skill succeeds. Our success region bounds the pose-estimate error instead, so a skill acts as a funnel [25] that the robot enters by localizing the part.

3 Problem Formulation

Inputs. The robot receives the K step drawings of a graphical assembly manual, such as an IKEA manual, CAD meshes of the parts P with their *attachment points*, and parameterized skills. Attachment points are connector locations such as dowel holes. A step that joins two or more subassemblies is an *assembly operation* h, where a single part counts as a subassembly. Each operation joins subassemblies built by earlier operations, so the operations form a tree. The *interpretation* of a step is the set of parts that it joins. The decoder (Sec. 4) estimates the tree $\hat{T}$ and, for each operation h, the probability m_h of a wrong interpretation.

Task-level actions. $\textsc{Assemble}_h$ runs the skill-level subtree of operation h. It retries *visible failures*, noticed at once, such as a dropped part. $\textsc{Inspect}_s$ checks the result with a sensor s. The visual inspection asks the VLM whether a camera image of the current assembly matches the step's manual drawing, and the force inspection pushes the placed part along its insertion axis. $\textsc{Disassemble}_h$ reverses a failed attempt. $\textsc{AskOperator}_h$ shows the operator four candidate interpretations of step h, that is, four candidate sets of parts that the step joins: the decoded set and the next three most probable sets. The operator selects the correct set or, if none of the four is correct, specifies it at the cost of a help request. $\textsc{RequestHelp}_h$ hands the operation to the operator. Every action has a cost, in units of one assembly operation without fasteners. Queries cost c_{ask} and help requests c_{help}.

Undetected failures. Other failures give no immediate signal, such as an unseated dowel. Such a failure stays *undetected* until an inspection detects it or a later operation reveals it. We distinguish three failure types f. A *connector failure* leaves a connector missing, unseated, or loose, with probability ε_κ per connector of type κ. An *orientation failure* mounts a part the wrong way round, with probability ε_o per attempt. A *misinterpretation* builds the wrong configuration in every attempt, and only the operator can correct it. An inspection with sensor s right after an operation detects a failure of type f with probability $d_s(f)$ and raises a false alarm with probability a_s. Once a later operation covers the failure, an inspection detects it only with a lower probability $\tilde{d}_s(f)$. A failure can also become visible at a later operation, e.g., when a part no longer fits. This happens with probability β_f, the *reveal rate*. A failure found later is repaired by disassembling back to its operation.

Problem 1 (Task-level BT synthesis)**.** *Given $\hat{T}$, the parameters above, and a risk bound δ, find*

$$B^\star \in \arg\min_{B \in \mathcal{F}} C(B) \quad \textit{subject to} \quad R(B) \le \delta. \tag{1}$$

Here B is a BT in the template family $\mathcal{F}$ (Sec. 5), $C(B)$ its expected cost, and the risk $R(B)$ *its expected number of undetected failures.*

Each undetected failure arises at one operation, so the risk of a tree is a sum over its operations. Since the probability of at least one undetected failure never exceeds their expected number, $R(B) \le \delta$ also bounds this probability. One error lies outside $R(B)$: a step decoded as joining no parts has no operation in $\hat{T}$, so the BT can neither inspect nor query it. With the 8B VLM, this happens to 9 of the 66 annotated operations, and the evaluation charges each as a help request.

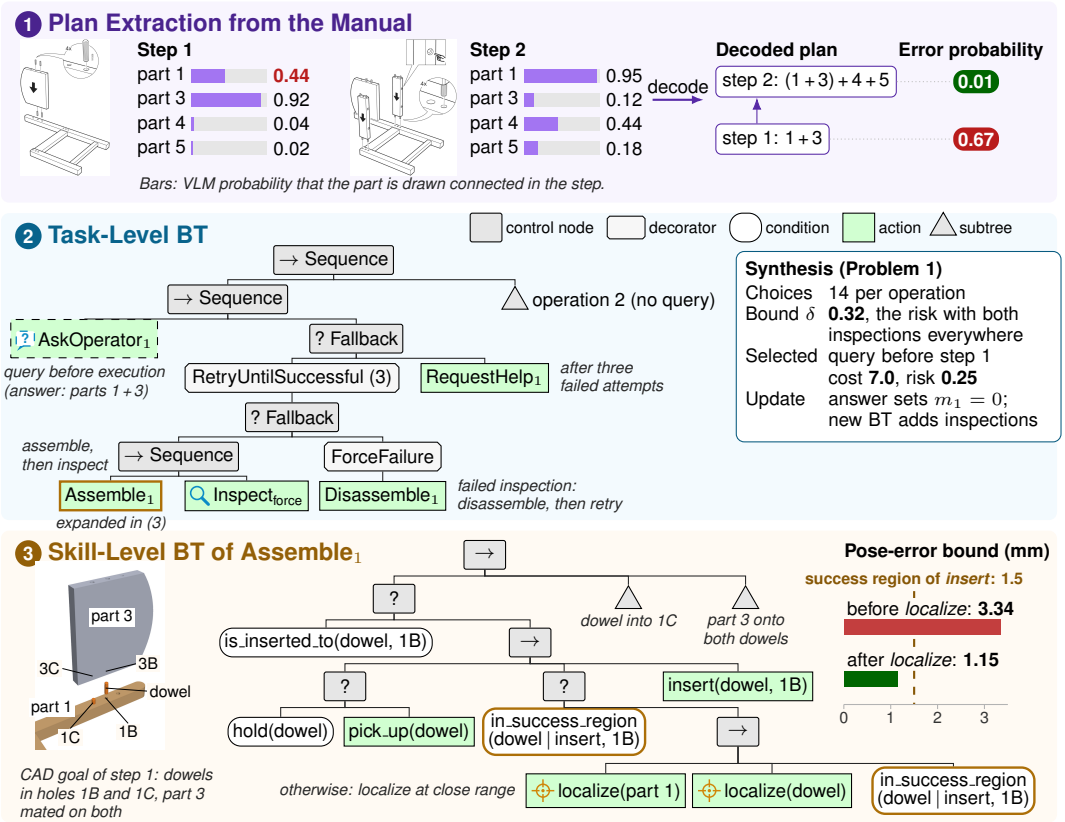


Figure 2: **Worked example on the IKEA SUNDVIK chair frame.** (1) VLM probability that each part is drawn connected in steps 1 and 2, and the decoded plan of the frame's four parts with each step's error probability. As the last step, step 2 must join all remaining parts, so its error probability is low. (2) Task-level BT as executed, built from BehaviorTree.CPP nodes. The box summarizes its synthesis. (3) Skill-level BT of Assemble$_1$, compiled by backward chaining from the CAD goal. The pose-error bound exceeds the success region of *insert*, so the BT first localizes part 1 and the dowel.

4 Probabilistic Task Model from Graphical Manuals

Questions to the VLM. A VLM asked for a whole plan returns one answer without a probability for each of its steps. We instead ask, for every step k and part i, whether part i is drawn connected to other parts. The VLM sees the step drawing, a labeled parts sheet [26], and a rendering of part i. Restricting the answer to the single token Yes or No lets us read the probability of Yes, $p_{k,i}$, from the output distribution, renormalized over the two tokens [27], without asking for a stated confidence.

Structured decoding. Let D_k be the set of parts drawn connected in step k. A subassembly is drawn as a whole, and subassemblies only merge. Hence D_k is either empty or a union of subassemblies that exist before step k (*partition consistency*). The product is complete after the last step, so the last step joins all remaining subassemblies (*completeness*). The plan is therefore a hidden Markov chain over partitions of P. Given all answers $p = (p_{k,i})$, the posterior of a plan is

$$\Pr(D_{1:K} \mid p) \;\propto\; \pi(D_{1:K}) \prod_{k=1}^{K} \prod_{i \in P} q_{k,i}^{[i \in D_k]} \, (1 - q_{k,i})^{[i \notin D_k]}, \qquad \operatorname{logit}(q_{k,i}) = \operatorname{logit}(p_{k,i})/\tau + b, \tag{2}$$

where $[\cdot]$ is 1 if its argument holds and 0 otherwise. The prior π favors steps that attach one new part to one subassembly, which yields linear assembly sequences [28]. The bias b corrects the VLM's tendency to answer No, and the temperature τ [29] compensates for correlated answers. We fit the prior weights, b, and τ by maximum likelihood on the annotated plans of the other 20 products. The

forward–backward algorithm [30] computes the posterior marginal $\Pr(D_k \mid p)$ of every step k. The Viterbi algorithm gives the most probable plan, whose non-empty steps are the operations of $\hat{T}$.

Error probabilities and prediction sets. The posterior marginal of a step gives the probability that each candidate interpretation is the correct one. The decoded interpretation $\hat{D}_h$ of operation h is therefore wrong with probability $m_h = 1 - \Pr(D_h = \hat{D}_h \mid p)$. As in KnowNo [18], the prediction set $\mathcal{R}_h$ holds the interpretations whose posterior exceeds a threshold. We choose the threshold on the other 20 products by split-conformal calibration [31], with miss rate $\alpha = 0.1$. On held-out products, the sets contain the correct interpretation for 89.6% of the operations.

5 Task-Level Behavior Tree Synthesis

BT templates. We use standard BT semantics [32] and BehaviorTree.CPP nodes [33]. For every operation h, a template makes two choices. The first is the inspection sequence σ_h: none, force, visual, force then visual, or visual then force. The second is when to query the operator: never, before the operation, or after a failed inspection. The subtree of h first builds its children's subtrees $\mathrm{ch}(h)$, then attempts the operation and runs the inspections. A failed inspection disassembles the attempt, and the attempt is retried. After $N = 3$ failed attempts, the subtree requests help:

$$
\begin{aligned}
\textsc{Sub}(h) &= \text{Sequence}\big(\textsc{Sub}(h')_{h' \in \mathrm{ch}(h)},\ [\textsc{Ask}_h]_{\text{before}},\ J_h\big),\\
J_h &= \text{Fallback}\big(\text{Retry}_N\big(\text{Fallback}(\text{Sequence}(\textsc{Assemble}_h, \textsc{Inspect}_{\sigma_h}),\ U_h)\big),\ \textsc{Help}_h\big),\\
U_h &= \text{ForceFailure}\big(\text{Sequence}\big(\textsc{Disassemble}_h,\ [\text{RunOnce}(\textsc{Ask}_h)]_{\text{after fail}}\big)\big),
\end{aligned}
$$

where $\textsc{Ask}_h$ and $\textsc{Help}_h$ abbreviate $\textsc{AskOperator}_h$ and $\textsc{RequestHelp}_h$, and a bracketed node is present only if the template queries there. Retry_N is the RetryUntilSuccessful decorator with N attempts. It, ForceFailure, and RunOnce are standard BehaviorTree.CPP decorators (Appendix B).

Dynamic program. The expected cost and the risk of a tree are sums of contributions from its operations. A subtree affects its ancestors only through the undetected failures it passes upward. Suppose a failure of type f has already passed the inspection types in a set S. A parent operation with inspection sequence σ catches it with probability

$$1 - (1 - \beta_f) \prod_{s \in \sigma \setminus S} \big(1 - \tilde{d}_s(f)\big). \tag{3}$$

A caught failure is repaired by disassembling back to the operation where it arose. Otherwise, it moves further up, having now passed the inspection types $S \cup \sigma$. A subtree thus depends on its ancestors only through these catch probabilities, one per failure type and passed set, which form its *inspection context*. Algorithm 1 runs bottom-up over $\hat{T}$. For every operation and inspection context, it computes the Pareto set of (cost, risk) of the subtree. The cheapest point within the bound gives $B^\star$.

Theorem 1. *Let $B \in \mathcal{F}$, assume A1–A5, and let visible failures occur with probability below one. (a) Every execution of B terminates with probability one. (b) Executing B in the task model induces a finite Markov chain whose expected cost, risk, and operator interactions equal those computed by Algorithm 1. (c) Algorithm 1 returns the exact Pareto set of $\mathcal{F}$. Hence $B^\star$ solves Problem 1.*

The main assumption (A1, Appendix D) is that failures at different operations are independent with known rates. The proof extends the Markov-chain analysis of stochastic BTs [34].

6 Skill-Level Subtrees with Measured Success Regions

Goal conditions from CAD. An operation joins a moving subassembly to a base through *mating pairs* of attachment points. We pair the attachment points of the two parts whose pairwise distances agree within 1 mm [35] and solve the pose in closed form [36]. The goal G_h of $\textsc{Assemble}_h$ uses the predicate *is_inserted_to* [3]. It requires every connector to be seated and the mating pairs to coincide.

Skills and success regions. A parameterized skill is a fixed controller with objects and bounded parameters [37, 38]. We use *pick_up*, *insert*, and *localize*, with the condition *hold*(x) for an object

Table 1: **Plan extraction (Q1)** on 21 products with 85 steps and 66 annotated operations. Majority vote keeps operations that appear in most of 10 sampled plans. No-join: accuracy on the 19 steps that join no parts. AUROC: separation of correct from wrong operations by confidence (single query: stated confidence / frequency in 10 sampled plans). Bold: best per VLM.

		Step exact	Operation				
Method	VLM	match ↑	Prec. ↑	Rec. ↑	F1 ↑	No-join ↑	AUROC ↑
Single-query plan (Manual2Skill prompt [1])	8B	34.1	28.6	21.2	24.3	78.9	0.55 / 0.79
Majority vote over 10 sampled plans	8B	42.4	40.0	30.3	34.5	**84.2**	0.76
Structured decoding (ours)	8B	**43.5**	**41.8**	**42.4**	**42.1**	47.4	**0.87**
Single-query plan	32B	**58.8**	**62.3**	**50.0**	**55.5**	**89.5**	0.75 / **0.87**
Structured decoding (ours)	32B	52.9	49.3	**50.0**	49.6	63.2	0.80

x in the gripper. *Insert* presses the held dowel onto the surface, moves it along an outward spiral around the estimated hole position, detects entry when the dowel drops into the hole, and detects seating from the axial force [39, 40]. The *success region* of a skill is the set of pose-estimate errors from which it succeeds with probability at least $p_{\min} = 0.8$. For *insert*, this set is described by a radius $r^\star$: the error of the estimated hole position plus the error of the estimated dowel-tip position, both measured perpendicular to the hole axis, must not exceed $r^\star$. The condition *in_success_region* holds when the robot's pose-error bound is at most $r^\star$. Backward chaining from G_h compiles the subtree (Fig. 2, panel 3). If *in_success_region* fails, it inserts *localize* to re-estimate the pose at close range. We measure $r^\star$ in simulation as the largest offset of the estimated hole position at which *insert* succeeds with probability at least $p_{\min}$, with 95% confidence [41, 42] (Appendix F).

7 Experiments

We ask one question per layer of the system. **Q1 (reading the manual):** the task-level planner can place queries well only if it knows which steps may be misread. Does the decoder recover the plan, and do its error probabilities match how often it is wrong? *Baselines:* one query for the whole plan with the Manual2Skill prompt [1], and a majority vote over 10 sampled plans. **Q2 (task-level BT):** inspections and operator queries take time, so they should be used only where failures are likely and hard to find later. Do the planned inspections and queries reduce undetected failures, and does the planner place them better than simple rules? *Baselines:* no prior system places both on a plan read from a manual, so we compare with common assembly-line inspection rules, three query rules including KnowNo [18], a planner that trusts the decoded plan, and the optimal adaptive policy. **Q3 (skill level):** the task-level BT is only as reliable as its skills. Do skills with measured success regions compose into a complete assembly under physics? *Ablation:* insertion without *localize*.

Setup. We use the VLM-planning benchmark of Manual2Skill++ [2]: 21 products (11 IKEA furniture items, 10 toy and mechanical models) with 85 annotated steps and 66 assembly operations. We read the manuals with Qwen3-VL-8B-Instruct and -32B-Instruct [43], used off the shelf without fine-tuning. Only the decoder's prior weights, bias, and temperature are fitted, on the other 20 products. For Q2, we use the plans decoded with the 8B VLM and run each task-level BT in simulated executions of the task model: failures, inspection outcomes, and operator answers are drawn at random, and the annotated plan decides whether a decoded interpretation is wrong. The costs and failure rates are assumed values (Appendix G). The visual-inspection rates are measured on rendered line drawings and are near chance for connectors. To test the task-level planner when its rates are wrong, the simulation uses 1.5 times the planner's failure probabilities ($\varepsilon_\kappa, \varepsilon_o$) and 0.8 times its reveal rates (β_f). Skill-level experiments run in MuJoCo [44] with a Franka Panda [45], a wrist camera, and a wrist force/torque sensor. The product is the SUNDVIK chair frame (four parts, six dowels; corrected CAD meshes, Appendix F). In these runs, the visual inspection checks the CAD goal conditions on the estimated part poses instead of querying the VLM.

Q1: With the 8B VLM, the decoder recovers twice as many operations as a single plan query, and its error probabilities match how often it is wrong. Of the 66 annotated operations, the

Table 2: **Task-level BTs (Q2)** on the decoded plans of 21 products, summed over products. Each baseline applies one rule to every operation. The right block gives the synthesized BT at the baseline's planned risk. Reduction: $1-$ its cost / baseline cost, 95% paired bootstrap interval. Costs include 20 for each of the 9 operations missing from the decoded plan. Cost: in units of one operation. Undet.: expected undetected failures. Inter.: expected queries, help requests, and later repairs. Myopic: query if $m_h c_{\text{help}} > c_{\text{ask}}$. KnowNo-style: query if the prediction set has several interpretations.

	Baseline			Synthesized BT at the baseline's risk		
Policy	Cost ↓	Undet. ↓	Inter. ↓	Cost ↓	Undet. ↓	Reduction
Inspection rules without queries						
No inspection	723.5	25.61	28.1	766.5	18.38	−6.0% [−20.7, 4.8]
Final inspection only	950.3	14.64	36.7	815.8	12.70	14.2% [2.9, 22.9]
Inspect every operation	1033.3	14.53	35.1	811.9	12.92	21.4% [11.5, 29.4]
Query rules with force inspection of every operation						
Always query	948.2	2.01	98.2	941.4	2.00	0.7% [0.1, 1.7]
KnowNo-style [18]	940.2	2.01	94.2	940.2	2.01	0.0% (same trees)
Myopic	923.9	2.93	85.2	922.1	3.00	0.2% [0.0, 0.4]
Task-level planner, minimizing expected cost $+ 20\times$ *risk*						
Trust the decoded plan ($m_h = 0$)	808.0	20.54	30.5			
Synthesized BT	900.8	3.96	84.7			

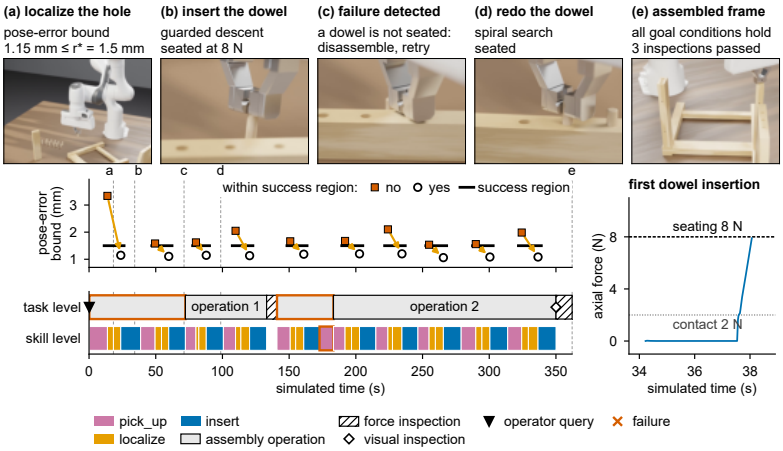


Figure 3: **One of the 40 runs on the SUNDVIK chair frame**, rendered from the recorded simulation. After one operator query, the BT detects two failed attempts, an unseated dowel and a failed grasp, and disassembles and redoes them without help. Middle: pose-error bound before (square) and after *localize* (circle) for each insertion, and actions over time. Right: axial force, first dowel insertion.

decoder recovers 42.4%, the single plan query 21.2%, and the majority vote 30.3% (operation recall, Table 1; 95% bootstrap interval of the advantage: 7.8 to 34.2 percentage points). On the 19 steps that join no parts, the decoder often joins parts one step early and is correct on 47.4% of them, against 78.9% for the single query. Its share of exactly decoded steps (43.5% against 34.1%) is therefore not significantly higher ($p = 0.38$, sign-flip test over products). The error probabilities rank operations well: a wrong operation has a higher error probability than a correct one with probability 0.87 (AUROC, 95% interval 0.76 to 0.95). They are also *calibrated*, i.e., correct on average: the mean error probability is 0.61, and 58% of the decoded operations are wrong. With the 32B VLM, the single plan query is more accurate than the decoder (F1 55.5% against 49.6%) but overconfident. The frequency of an operation in 10 sampled plans implies that 23% of its operations are wrong, whereas 38% are. The decoder implies 52%, and 51% are wrong. *Takeaway:* the decoder's error probabilities are calibrated with both VLMs, and with the 8B VLM it also recovers more operations.

Q2: Querying where the decoder is unsure removes most undetected failures, and the synthesized BT costs less than fixed inspection rules. *Does querying help?* Summed over the 21 products,

executing the decoded plan without queries, as current pipelines do, leaves 20.5 expected undetected failures (Table 2). The synthesized BT leaves 4.0, and the three query rules leave 2.0 to 2.9. *Does the synthesis beat simple rules?* For each rule, we set the risk bound of the task-level planner to the risk that the rule plans for, and compare costs (right block of Table 2). The synthesized BT then costs 14.2% less than inspecting only the finished product and 21.4% less than inspecting every operation, because it queries the likely misread steps and inspects each joint before a later part covers it. Against no inspection, its cost is 6.0% higher, but the 95% interval (-20.7% to 4.8%) includes zero. Against the query rules, which use the same error probabilities and prediction sets, it saves at most 0.7%. Since a query costs a tenth of a help request, a query pays off whenever $m_h > 0.1$. This holds for 81% of the operations, and the rules query all of them. The saving grows with the query cost: over 180 settings of failure, reveal, and detection rates, the median saving over the myopic rule is 0.1 to 0.3% when a query costs at most a quarter of a help request, and 4.9% when it costs half (Table 6). Replacing the per-step error probabilities by their mean changes the expected cost by only 1.2 units out of about 890 (Appendix G). Against the optimal adaptive policy, the best BT costs a median of at most 2.8% more at the same risk (17 products, Appendix E). *Takeaway:* calibrated error probabilities make querying effective and carry most of the gain, and the synthesis beats fixed inspection rules.

Q3: In simulation, the hierarchical BT completes the chair frame in 35 of 39 runs, and no failure stays undetected. Both success regions were confirmed at $r^\star = 1.5$ mm (Table 3). The pose-error bounds of the initial estimates are larger (3.34 mm for a dowel, 3.77 mm for part 3), so the BT ran *localize* before each of the 311 insertions. Without *localize*, the dowel insertion still succeeded in 59 of 60 trials, because the actual error happened to be below $r^\star$ in 56 of them. The BT cannot know this in advance. Of 40 runs from the manual to the assembled frame, one crashed the simulator. Of the other 39, 35 completed the frame (90%, 95% interval 76 to 97%), and 20 of these needed no help request. In 6 completed runs, the BT detected 7 failed attempts and redid them without help (Fig. 3). The 4 unfinished runs stopped because the simulator cannot disassemble a fixed part. No run ended with an undetected failure. The task-level planner predicted 0.25 per run (9.6 over the 39 runs), a conservative value from the 95% upper bounds of the failure rates. *Takeaway:* skills with measured preconditions compose into a full assembly, and every failure was detected.

8 Limitations

Q2 samples executions from the task-level planner's own model with rescaled rates, so it does not test correlated failures, such as a misread step that changes every later subassembly, or failed repairs. The one structural change we tested, a covered failure that fails a later inspection instead of being repaired at its origin, raised the cost by 1.1% (Appendix D). Statistics logged in physics executions would test this further. Most costs and failure rates are assumed, and the gain over query rules depends on them: the synthesized BT saves at most 1.0% over the myopic and KnowNo-style rules whenever a query costs at most a quarter of a help request (Table 6). Measured operator times would replace these assumptions for a given workcell. Steps decoded as joining nothing also lie outside the risk bound. Adding each such step's probability of being an operation, $1 - \Pr(D_k = \emptyset \mid p)$, to the risk and allowing a query on it would close this gap. The skill level was tested on one product in simulation, with compliant contact, bounded perception noise, pose-based visual inspection, and no disassembly of fixed parts. Since it needs only CAD meshes with attachment points, the next tests are other IKEA items, stiffer contact, a learned pose estimator [46], and a real robot.

9 Conclusion

We presented a system that decides where a robot should inspect and when it should query an operator, when both the plan read from a manual and its execution may be wrong. Querying the operator about uncertain steps reduces the expected undetected failures over 21 products from 20.5 to 4.0, and in simulation the system completes a chair frame in 35 of 39 runs. For planning with learned abstractions, the planner needs calibrated error probabilities of the foundation model. Per-step values pay off beyond their mean only under a risk bound with expensive queries.

References

[1] C. Tie, S. Sun, J. Zhu, Y. Liu, J. Guo, Y. Hu, H. Chen, J. Chen, R. Wu, and L. Shao. Manual2Skill: Learning to read manuals and acquire robotic skills for furniture assembly using vision-language models. In *Robotics: Science and Systems (RSS)*, 2025. doi:10.15607/RSS.2025.XXI.150.

[2] C. Tie, S. Sun, Y. Lin, Y. Wang, Z. Li, Z. Zhong, J. Zhu, Y. Pang, H. Chen, J. Chen, R. Wu, and L. Shao. Manual2Skill++: Connector-aware general robotic assembly from instruction manuals via vision-language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2026. URL https://arxiv.org/abs/2510.16344.

[3] J. Ao, F. Wu, Y. Wu, A. Swikir, and S. Haddadin. LLM-as-BT-Planner: Leveraging LLMs for behavior tree generation in robot task planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1233–1239, 2025. doi:10.1109/ICRA55743.2025.11128454.

[4] J. Styrud, M. Iovino, M. Norrlöf, M. Björkman, and C. Smith. Automatic behavior tree expansion with LLMs for robotic manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1225–1232, 2025. doi:10.1109/ICRA55743.2025.11127942.

[5] X. Zhao, Y. Wang, Y. Wu, F. Wu, T. Sun, Z. Miao, S. Haddadin, and A. Knoll. Video-to-BT: Generating reactive behavior trees from human demonstration videos for robotic assembly, 2025. URL https://arxiv.org/abs/2509.16611. arXiv preprint arXiv:2509.16611.

[6] C. L. Shek, Y. Wang, J. Liu, K. Suzuki, P. Tokekar, and T. Koike-Akino. Read the manual: Grounding behavior tree synthesis for multi-machine factory operation. Technical Report TR2026-149, Mitsubishi Electric Research Laboratories, 2026. URL https://www.merl.com/publications/TR2026-149. IEEE/RSJ IROS 2026 Workshop.

[7] F. Ahmad, H. Ismail, J. Styrud, M. Stenmark, and V. Krueger. A unified framework for real-time failure handling in robotics using vision-language models, reactive planner and behavior trees. *arXiv preprint arXiv:2503.15202*, 2025. URL https://arxiv.org/abs/2503.15202.

[8] Z. Navasardyan, T. Danielyan, and H. Davtyan. FailBench: How reliable are VLMs at judging robot task success? *arXiv preprint arXiv:2609.03611*, 2026. URL https://arxiv.org/abs/2609.03611.

[9] Y. Tian, J. Jacob, Y. Huang, J. Zhao, E. L. Gu, P. Ma, A. Zhang, F. Javid, B. Romero, S. Chitta, S. Sueda, H. Li, and W. Matusik. Fabrica: Dual-arm assembly of general multi-part objects via integrated planning and learning. In *Conference on Robot Learning (CoRL)*, volume 305 of *PMLR*, pages 406–419, 2025. URL https://proceedings.mlr.press/v305/tian25a.html.

[10] H. Kurniawati, D. Hsu, and W. S. Lee. SARSOP: Efficient point-based POMDP planning by approximating optimally reachable belief spaces. In *Robotics: Science and Systems (RSS)*, 2008. doi:10.15607/RSS.2008.IV.009.

[11] A. Somani, N. Ye, D. Hsu, and W. S. Lee. DESPOT: Online POMDP planning with regularization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2013. URL https://papers.nips.cc/paper/2013/hash/c2aee86157b4a40b78132f1e71a9e6f1-Abstract.html.

[12] P. Santana, S. Thiébaux, and B. Williams. RAO*: An algorithm for chance-constrained POMDP's. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016. doi:10.1609/aaai.v30i1.10423.

[13] G. F. Lindsay and A. B. Bishop. Allocation of screening inspection effort—a dynamic-programming approach. *Management Science*, 10(2):342–352, 1964. doi:10.1287/mnsc.10.2.342.

[14] R. R. Britney. Optimal screening plans for nonserial production systems. *Management Science*, 18(9):550–559, 1972. doi:10.1287/mnsc.18.9.550.

[15] Z. Shi and P. Sandborn. Optimization of test/diagnosis/rework location(s) and characteristics in electronic system assembly. *Journal of Electronic Testing*, 22(1):49–60, 2006. doi:10.1007/s10836-006-6593-3.

[16] H. Rau, Y.-H. Chu, and K.-H. Cho. Layer modelling for the inspection allocation problem in re-entrant production systems. *International Journal of Production Research*, 43(17):3633–3655, 2005. doi:10.1080/13629390500124515.

[17] C. Boutilier. Approximately optimal monitoring of plan preconditions. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 54–62, 2000. URL https://arxiv.org/abs/1301.3839.

[18] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley, Z. Xu, D. Sadigh, A. Zeng, and A. Majumdar. Robots that ask for help: Uncertainty alignment for large language model planners. In *Conference on Robot Learning (CoRL)*, volume 229 of *PMLR*, pages 661–682, 2023. URL https://proceedings.mlr.press/v229/ren23a.html.

[19] K. Liang, Z. Zhang, and J. F. Fisac. Introspective planning: Aligning robots' uncertainty with inherent task ambiguity. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2402.06529.

[20] J. F. Mullen, Jr. and D. Manocha. LBAP: Improved uncertainty alignment of LLM planners using Bayesian inference, 2024. URL https://arxiv.org/abs/2403.13198. arXiv preprint arXiv:2403.13198.

[21] Z. Liu, A. Bahety, and S. Song. REFLECT: Summarizing robot experiences for failure explanation and correction. In *Conference on Robot Learning (CoRL)*, volume 229 of *PMLR*, pages 3468–3484, 2023. URL https://proceedings.mlr.press/v229/liu23g.html.

[22] G. Konidaris, L. P. Kaelbling, and T. Lozano-Pérez. From skills to symbols: Learning symbolic representations for abstract high-level planning. *Journal of Artificial Intelligence Research*, 61:215–289, 2018. doi:10.1613/jair.5575.

[23] Z. Wang, C. R. Garrett, L. P. Kaelbling, and T. Lozano-Pérez. Learning compositional models of robot skills for task and motion planning. *The International Journal of Robotics Research*, 40(6–7):866–894, 2021. doi:10.1177/02783649211004615.

[24] N. Gandotra, I. Mishani, L. Yuan, O. Salzman, and M. Likhachev. Constant-time planning for chaining collision-free motion to manipulation behaviors, 2025. URL https://arxiv.org/abs/2512.00939. arXiv preprint arXiv:2512.00939.

[25] R. R. Burridge, A. A. Rizzi, and D. E. Koditschek. Sequential composition of dynamically dexterous robot behaviors. *The International Journal of Robotics Research*, 18(6):534–555, 1999. doi:10.1177/02783649922066385.

[26] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. *arXiv preprint arXiv:2310.11441*, 2023. URL https://arxiv.org/abs/2310.11441.

[27] S. Kadavath et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022. URL https://arxiv.org/abs/2207.05221.

[28] L. S. Homem de Mello and A. C. Sanderson. AND/OR graph representation of assembly plans. *IEEE Transactions on Robotics and Automation*, 6(2):188–199, 1990. doi:10.1109/70.54734.

[29] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1321–1330, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

[30] L. R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi:10.1109/5.18626.

[31] V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic Learning in a Random World*. Springer, 2005. doi:10.1007/b106715.

[32] M. Colledanchise and P. Ögren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018. doi:10.1201/9780429489105.

[33] D. Faconti. BehaviorTree.CPP: Behavior trees library in C++, version 4. https://github.com/BehaviorTree/BehaviorTree.CPP, 2024.

[34] M. Colledanchise, A. Marzinotto, and P. Ögren. Performance analysis of stochastic behavior trees. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3265–3272, 2014. doi:10.1109/ICRA.2014.6907328.

[35] H. Yang, J. Shi, and L. Carlone. TEASER: Fast and certifiable point cloud registration. *IEEE Transactions on Robotics*, 37(2):314–333, 2021. doi:10.1109/TRO.2020.3033695.

[36] S. Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4):376–380, 1991. doi:10.1109/34.88573.

[37] B. C. da Silva, G. Konidaris, and A. Barto. Learning parameterized skills. In *International Conference on Machine Learning (ICML)*, 2012. URL https://arxiv.org/abs/1206.6398.

[38] M. Dalal, D. Pathak, and R. Salakhutdinov. Accelerating robotic reinforcement learning via parameterized action primitives. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2110.15360.

[39] A. Rodriguez, D. Bourne, M. T. Mason, G. F. Rossano, and J. Wang. Failure detection in assembly: Force signature analysis. In *IEEE International Conference on Automation Science and Engineering (CASE)*, pages 210–215, 2010. doi:10.1109/COASE.2010.5584452.

[40] M. Noseworthy, B. Tang, B. Wen, A. Handa, C. C. Kessens, N. Roy, D. Fox, F. Ramos, Y. Narang, and I. Akinola. FORGE: Force-guided exploration for robust contact-rich manipulation under uncertainty. *IEEE Robotics and Automation Letters*, 10(5):4436–4443, 2025. doi:10.1109/LRA.2025.3551637.

[41] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi:10.2307/2331986.

[42] W. Maurer, L. A. Hothorn, and W. Lehmacher. Multiple comparisons in drug clinical trials and preclinical assays: A-priori ordered hypotheses. In J. Vollmar, editor, *Biometrie in der chemisch-pharmazeutischen Industrie*, volume 6, pages 3–18. Fischer, Stuttgart, 1995.

[43] S. Bai et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025. URL https://arxiv.org/abs/2511.21631.

[44] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012. doi:10.1109/IROS.2012.6386109.

[45] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors. MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo. https://github.com/google-deepmind/mujoco_menagerie, 2022.

[46] B. Wen, W. Yang, J. Kautz, and S. Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 17868–17879, 2024. doi:10.1109/CVPR52733.2024.01692. URL https://arxiv.org/abs/2312.08344.

[47] E. Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

[48] I. F. Jasim, P. W. Plapper, and H. Voos. Position identification in force-guided robotic peg-in-hole assembly tasks. *Procedia CIRP*, 23:217–222, 2014. doi:10.1016/j.procir.2014.10.077.

[49] Y. Narang, K. Storey, I. Akinola, M. Macklin, P. Reist, L. Wawrzyniak, Y. Guo, A. Moravanszky, G. State, M. Lu, A. Handa, and D. Fox. Factory: Fast contact for robotic assembly. In *Robotics: Science and Systems (RSS)*, 2022. doi:10.15607/RSS.2022.XVIII.035.

[50] Franka Robotics GmbH. libfranka: Cartesian impedance control example. https://github.com/frankaemika/libfranka, 2026. File examples/cartesian_impedance_control.cpp, accessed Oct. 2026.

[51] A. Gotovos, N. Casati, G. Hitz, and A. Krause. Active learning for level set estimation. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1344–1350, 2013. URL https://www.ijcai.org/Abstract/13/202.

[52] B. Tang, I. Akinola, J. Xu, B. Wen, A. Handa, K. Van Wyk, D. Fox, G. S. Sukhatme, F. Ramos, and Y. Narang. AutoMate: Specialist and generalist assembly policies over diverse geometries. In *Robotics: Science and Systems (RSS)*, 2024. doi:10.15607/RSS.2024.XX.064.

[53] S. S. Wilks. Determination of sample sizes for setting tolerance limits. *The Annals of Mathematical Statistics*, 12(1):91–96, 1941. doi:10.1214/aoms/1177731788.

[54] M. Heo, Y. Lee, D. Lee, and J. J. Lim. FurnitureBench: Reproducible real-world benchmark for long-horizon complex manipulation. In *Robotics: Science and Systems (RSS)*, 2023. doi:10.15607/RSS.2023.XIX.041.

[55] R. Wang, Y. Zhang, J. Mao, R. Zhang, C.-Y. Cheng, and J. Wu. IKEA-Manual: Seeing shape assembly step by step. In *NeurIPS Datasets and Benchmarks Track*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/b645d1a085bcb39bece5c03703b62464-Abstract.html.

[56] W. E. Hick. On the rate of gain of information. *Quarterly Journal of Experimental Psychology*, 4(1):11–26, 1952. doi:10.1080/17470215208416600.

A Decoder

Prior, bias, and temperature. The prior π of Eq. (2) factorizes over steps. A step that joins the set u of subassemblies scores $-\lambda_{\text{units}}(|u| - 2) - \lambda_{\text{sub}} \max(0,\, n_{>1}(u) - 1)$, where $n_{>1}(u)$ is the number of joined subassemblies with more than one part. A step that joins nothing before the product is complete scores $-\lambda_{\text{prep}}$. The weights, the bias b, and the temperature $\tau \in \{2^{j/2} : j = -4, \ldots, 6\}$ maximize the log-likelihood of the annotated plans of the other 20 products. The decoded plan uses $\tau = 1$, and the error probabilities and prediction sets use the fitted τ.

Inference. For a given partition, the score of a candidate set decomposes over subassemblies. Candidates are therefore enumerated best-first. Sets and partitions with less than e^{-12} of the best probability are pruned. At most 512 sets per partition and 2048 partitions per step are kept. Answer

probabilities are clipped to $[10^{-4}, 1-10^{-4}]$. Forward–backward and Viterbi agree with brute-force enumeration on random instances with four parts and three steps.

Prediction sets. The score of an operation is $1 - \Pr_\tau(\text{annotated interpretation})$. The threshold is its split-conformal quantile over the operations of the other 20 products, with $\alpha = 0.1$. Because π, b, and τ are fitted on the same products, the guarantee is approximate. These sets cover 89.6% of the operations and all operations of 16 of the 21 products. A score that uses no annotations restores the guarantee if products are exchangeable. It sets π uniform, $b = 0$, and $\tau = 1$, and it takes the largest score over the decoded operations of a product. It covers all operations of 19 products, over the 51 operations decoded with this setting. The four interpretations shown in a query cover 67.2% of the operations.

Metrics. The interpretation of step k is the union of the subassemblies that D_k joins if there are at least two, and none otherwise. A decoded operation is correct if the annotated plan has an operation at the same step with the same parts. The decoder predicts 67 operations for the 66 annotated ones, and 28 are correct. Precision is therefore 28/67 and recall 28/66. Intervals resample products (10^4 resamples, percentile). The stated-confidence AUROC of the 8B single query uses 20 products, because the stated confidences for one product could not be parsed.

Ablations. Operation F1 is 36.7% with a uniform prior, 37.3% without the bias, 31.2% without partition consistency, and 18.8% without the completeness constraint, against 42.1% for the full decoder. With $\tau = 1$, F1 is unchanged, but the mean error probability drops to 0.38 against an error rate of 0.58. The expected calibration error of the full decoder is 0.130 [0.091, 0.249] over ten equal-width bins. With the 32B VLM, the decoder reaches an F1 score of 49.6% and an AUROC of 0.80. Its mean error probability is 0.52, and its error rate 0.51. The visual-inspection rates use 389 of 393 rendered drawings. The four excluded drawings belong to one step whose reference image shows only a fragment of a part.

Prompts. Every closed question is answered with one token restricted to the options, and option probabilities are renormalized over the options. Part question (step drawing, parts sheet, rendering of part i): *"The first image is step k of a graphical assembly manual. The second image is the parts sheet: every part of the product, each labeled with its number. The third image shows part i on its own. In the manual, parts may be drawn rotated, at a different scale, or partly hidden by other parts. Is part i drawn connected to other parts in the step image, either already assembled with them or being attached to them? Answer No if part i is not drawn, or is drawn lying on its own. Answer Yes or No."* Visual inspection (manual drawing, line drawing of the current assembly): *"Does the current assembly match the manual (same parts, attached in the same places and orientations)? Answer Yes or No."* The single-query baseline uses the planning prompt of Manual2Skill [1], with the parts sheet in place of the scene photograph. Its answer is then parsed into a plan.

B Behavior-tree node semantics

We use BehaviorTree.CPP v4 [33]. *Sequence* restarts from its first child after a child fails, so a failed inspection inside Retry_N leads to DISASSEMBLE_h and a new ASSEMBLE_h. *SequenceWithMemory* would instead resume at the failed child and re-run the inspection without redoing the operation. *ReactiveSequence* re-ticks earlier children on every tick. *RetryUntilSuccessful* ticks its child at most N times. *ForceFailure* returns FAILURE once its child finishes. *RunOnce* executes its child on the first tick and is skipped afterwards. ASKOPERATOR_h writes the operator's answer to the blackboard entry of step h, which the other actions of h read.

C Synthesis algorithm

Algorithm 1 Task-level BT synthesis (dynamic program over $\hat{T}$)

function SOLVE(h, w) ▷ w: inspection context of h; memoized on (h, w)
  $\mathcal{A} \leftarrow \emptyset$
  for all local choices (σ, q) of h **do** ▷ inspection sequence σ, query option q
    $\mathcal{S} \leftarrow \{(c_h, r_h)\}$ ▷ cost and risk of operation h under (σ, q) and w
    for all children u of h **do**
      $\mathcal{S} \leftarrow \textsc{Prune}\big(\mathcal{S} \oplus \textsc{Solve}(u, \textsc{ChildContext}(w, \sigma, h))\big)$ ▷ Eq. (3)
    $\mathcal{A} \leftarrow \mathcal{A} \cup \mathcal{S}$
  return PRUNE($\mathcal{A}$) ▷ Pareto-optimal points
$\mathcal{A}_0 \leftarrow \bigoplus_{\text{roots } r} \textsc{Solve}(r, w_0)$; $B^\star \leftarrow$ cheapest point in $\mathcal{A}_0$ within the bound, compiled into a BT

Template family. Five inspection sequences and three query options give 15 combinations. Asking after a failed inspection needs an inspection, which leaves 14 choices per operation. A plan with $|\hat{T}|$ operations thus has $14^{|\hat{T}|}$ trees. They include common hand-designed rules, such as inspecting every operation or only the finished product.

Size and time. The decoded plans have depth at most five and at most 25 inspection contexts per operation, and the largest Pareto set has 91 points. Synthesis takes a median of 0.14 s per product (at most 0.71 s). We enumerated $\mathcal{F}$ exhaustively on the 17 plans with up to four operations (171,136 trees). On the 13 plans with up to three operations, enumeration returns the same Pareto sets as Algorithm 1. On the four plans with four operations, it returns the same optimal cost at three risk bounds.

D Proofs

Assumptions in full. (A1) Undetected failures and inspection outcomes at different operations are independent, and the parameters are known. Every carried failure receives its own detection draw per inspection type. (A2) An inspection type that a failure has passed does not detect it again. Covered failures are detected with probability $\tilde{d}_s(f)$, and the visual verdict on a misinterpretation is fixed once drawn. (A3) $\textsc{Assemble}_h$ inspects the failures carried from its children once per inspection type in σ_h. A carried failure that is revealed or detected is repaired at its origin. (A4) Visible failures are handled within $\textsc{Assemble}_h$, every flagged attempt is disassembled, and disassembly and help requests succeed. (A5) An operator correction resolves the misinterpretation. The model also charges c_{ask} per query without re-planning, and it treats misreadings of different steps as independent, although a misread step changes later subassemblies.

Throughout, assumptions A1–A5 above hold, and all statements refer to the decoded plan $\hat{T}$ for which B was synthesized.

Model state. An episode of $B \in \mathcal{F}$ has a state with four components. (1) The active child of every composite node. (2) The attempt counter of every retry decorator ($\leq N$) and the status of every RunOnce node. (3) For every operation h: whether its interpretation is wrong, the stored visual verdict on a wrong interpretation, and the execution failure of the current attempt. A wrong interpretation is sampled once with probability m_h and cleared by $\textsc{AskOperator}_h$ or $\textsc{RequestHelp}_h$. (4) The set of undetected failures, each with its type, its operation of origin, its passed inspection set, and the operations built on top of it so far. All components take finitely many values. Leaves are actions that return SUCCESS or FAILURE. By A1 and A2, the next state depends only on the current state and on independent random draws. Ticking B thus induces a Markov chain on a finite state space.

Theorem 1(a) (termination). A visible failure of $\textsc{Assemble}_h$ is retried inside the action. Visible failures occur with probability $p_{\text{obs}} < 1$, so the number of such retries is geometric and hence finite

almost surely and in expectation. Late repairs (A3) are finite sequences of operations. An attempt executes $\textsc{Assemble}_h$ and at most $|\sigma_h|$ inspections, and a failed inspection ends it. $\textsc{Disassemble}_h$ then runs, together with $\textsc{AskOperator}_h$ once under RunOnce, before ForceFailure reports the failure to the retry decorator. The decorator starts at most N attempts, and if all fail, $\textsc{RequestHelp}_h$ completes the operation. In the model, corrections are applied in place (A5). No leaf of a subtree's outer Sequence fails, so each subtree is executed once. At most $\sum_h \big(N(2 + |\sigma_h|) + 2\big)$ task-level actions are therefore executed. With geometric retries inside each Assemble, every episode terminates with probability one, although the number of skill-level attempts is not bounded. During execution, a correction of the set of parts, which A5 does not cover, ends the tree, and the executive re-synthesizes. Each such correction fixes the interpretation of one more step, so at most K revisions occur.

Per-operation recursion. Consider an operation h with local policy (σ, q), where $q \in \{\text{never}, \text{before}, \text{on flag}\}$ is the query option. Let $\mu \in \{0, 1\}$ indicate a wrong interpretation. It is drawn with probability m_h if $q \neq \text{before}$, and it is set to 0 at cost c_{ask} if $q = \text{before}$. Let $V(i, g, \mu, \nu)$ be the expected cost from attempt i onward, together with the expected undetected failures left at h. Here $g \in \{\text{unknown}, \text{flag}, \text{pass}\}$ is the stored visual verdict on a wrong interpretation, and ν records whether the operator has been queried after a flag. If $i = N$, V equals c_{help} and leaves no failure. Otherwise, the attempt costs one operation, inflated by the visible-failure retries, plus every inspection it reaches. The visual verdict on a wrong interpretation is drawn once and then reused. An execution failure is drawn fresh, with probability $1 - \prod_\kappa (1 - \varepsilon_\kappa)^{n_\kappa}$ for a connector failure and ε_o for an orientation failure. The inspections are evaluated in the order of σ, with fresh outcomes except for the stored verdict. False alarms occur only on attempts without a fault. If all inspections pass, the present failures remain at h with passed set σ. Otherwise, the attempt is disassembled at cost $c_{\text{dis}}(h)$, and $V(i + 1, \cdot)$ is added. If in addition $q = \text{on flag}$ and $\nu = 0$, the operator is queried at cost c_{ask}, which sets $\mu = 0$, $g = \text{unknown}$, and $\nu = 1$. Averaging $V(0, \text{unknown}, \mu, 0)$ over μ gives the local cost and the failures created at h.

Theorem 1(b) (exact evaluation). By linearity of expectation, the expected cost is the sum over operations of the expected cost incurred while J_h is active. The expected number of undetected failures is the sum, over all failures created, of the probability that each one survives to the end. While J_h is active, two contributions arise, and A1 makes them independent.

(i) By A3, each failure carried from a child subtree is inspected once during the execution of J_h. Every inspection of σ_h runs in the accepted attempt, so each inspection type examines it. The failure is revealed with probability β_f. Otherwise, each inspection type in σ_h that it has not yet passed detects it with probability $\tilde{d}_s(f)$. A found execution failure is repaired at cost $c_{\text{dis}}(h)$ plus its accumulated redo cost, and the repair leaves an undetected failure with probability ε_r. A found misinterpretation costs $c_{\text{dis}}(h) + c_{\text{help}}$. A failure that is not found continues upward with passed set $S \cup \sigma_h$. Unless it is a misinterpretation, its redo cost grows by $c_{\text{dis}}(h) + c_{\text{asm}}(h)$. The model applies this inspection even if J_h ends in a help request.

(ii) The per-operation recursion gives the failures created at h and the local cost.

Both contributions are linear in the expected counts and accumulated redo costs of the carried failures. Composing (i) and (ii) from the leaves to the root reproduces the values computed by Algorithm 1. Therefore $C(B)$ and $R(B)$ equal the expectations of the Markov chain. The expected number of operator interactions is additive in the same way. Queries and help requests at h enter the per-operation recursion. A misinterpretation found at an ancestor enters through a weight that depends only on the ancestors' inspection sets, like the risk weight. These values are also checked independently, by exhaustive enumeration of $\mathcal{F}$ on small plans and by Monte Carlo execution of the compiled trees.

Theorem 1(c) (optimality within the risk bound). Fix the local policies of all ancestors of h. By (i), every failure that the subtree of h passes upward adds to the total cost and risk through fixed coefficients. These coefficients depend only on the ancestors' inspection sets and on disassembly

and operation costs, and not on the ancestors' query options. They form the inspection context w of Sec. 5. The local contribution of h does not depend on the policies of its children. The subtree's contribution is therefore a point (c, r) determined by its own policies and by w, and the total is the sum over disjoint subtrees. Let $\mathcal{S}_h(w)$ be the set of points achievable by the subtree of h, and let $\mathrm{Par}(\cdot)$ denote the Pareto subset. For sums of independent choices, $\mathrm{Par}(\mathcal{A} \oplus \mathcal{B}) \subseteq \mathrm{Par}(\mathcal{A}) \oplus \mathrm{Par}(\mathcal{B})$. Indeed, if $a + b$ were non-dominated and a' dominated a, then $a' + b$ would dominate $a + b$. Likewise, $\mathrm{Par}(\bigcup_\pi \mathcal{S}_\pi) \subseteq \bigcup_\pi \mathrm{Par}(\mathcal{S}_\pi)$ over the local policies π of h. By induction from the leaves, $\textsc{Solve}(h, w)$ returns $\mathrm{Par}(\mathcal{S}_h(w))$. At the root, it returns the Pareto set of $\mathcal{F}$, and for a forest the root sets are combined in the same way. The cheapest point with risk at most δ is therefore optimal for (1). It exists if and only if δ is at least the minimum achievable risk. Every member of $\mathcal{F}$ is dominated by or equal to a returned point.

Complexity. Let $L = 14$ be the number of distinct local policies and M the largest Pareto set encountered. SOLVE is called once per operation and inspection context. The inspection context of h depends only on the inspection sets chosen by its ancestors. There are at most $4^{\mathrm{depth}(h)}$ of them, because the order of the inspections does not change passed sets. Each call costs $O(L \cdot m \cdot M^2)$ for m children. For chain-shaped plans, the number of inspection contexts grows exponentially with depth.

Exact chance constraint. Let $w_k(h)$ be the probability that a failure left at h with key $k = (f, S)$ ends undetected. It follows the recursion of the inspection context, with $w = 1$ above the root. Each failure produces at most one undetected failure. By A1, the failures left at different operations end undetected independently. Hence $\Pr(\text{no undetected failure}) = \prod_h q_h(w(h))$, with $q_h(w) = \sum_L P_h(L) \prod_{k \in L}(1 - w_k)$, where $P_h(L)$ is the probability that the accepted attempt of h leaves exactly the failure set L. The term $-\log \Pr(\text{no undetected failure})$ is therefore additive over operations and depends on the ancestors only through the inspection context. The proof of Theorem 1(c) then applies to the pair (cost, $-\log \Pr$). The exact probability of no undetected failure agrees with Monte Carlo estimates on six products ($|z| \leq 1.5$).

Executing the compiled BT. The task model repairs carried failures at their origin. A simulator that ticks the compiled BT instead lets a carried failure fail a parent's inspection, which triggers disassembly and retries. Over the 21 products, the synthesized trees then cost 664.3 $\pm$ 1.0 against the 657.0 computed by Algorithm 1, with risk 2.359 against 2.351.

E Comparison with an adaptive policy

The constrained POMDP has the same costs, failure rates, and inspection model as the dynamic program. Its state holds the exact posterior over the failure status of every completed operation, and the attempt counter, inspections run, and query state of the current operation. After every observation, the policy may assemble, inspect with either sensor, accept, disassemble and retry, query the operator, or request help. Restricted to $\mathcal{F}$, its values match the dynamic program to within 10^{-10} on 323 trees from 17 products. For one or two operations, the model has up to 2.9 million states and is solved exactly by its occupancy-measure linear program [47]. For three or four operations, we obtain a lower bound on the optimal cost by telling the policy the failure status of completed operations. Where both are computed, this bound lies a median of 0.11% (at most 1.21%) below the exact optimum. Over the 63 pairs of product and risk bound solved exactly, the best BT costs a median of 2.8% more than the optimal adaptive policy (at most 26.0%, on a product with one operation). Against the lower bound for three or four operations, the median is 0.8% (at most 8.7%), so the true gap there is smaller. In the exact solutions, decisions that depend on outcomes at other operations carry 0.5% of the probability of the optimal policy. Plans with five or six operations are not covered.

F Skill level

Placement. Two mating pairs whose distances and normal angles agree within $\epsilon_t = 1\,\mathrm{mm}$ and $\epsilon_R = 0.05\,\mathrm{rad}$ define a pose of the moving subassembly, computed in closed form [36]. At that

pose, all attachment points that coincide with opposite normals are mated, and the pose is refined. A pose is rejected if it leaves a free motion or a residual exceeds the tolerance. It is also rejected if the moved part intersects a placed part by more than $\delta_g = 1$ mm. Hole axes are fitted to the meshes. The SUNDVIK meshes and attachment points of the benchmark conflict, and each conflict is corrected by the smallest edit derived from the meshes: the backrest and the rails differ in length by 10 mm, the frame holes for the backrest are 14.7 mm from its pilot holes, and three hole spacings disagree by 1–2 mm.

Goal conditions. A connector of length ℓ seated in a hole of depth d_a protrudes $\ell - \min(d_a, \ell)$, which must not exceed the depth of the mating hole. All mating pairs of an operation must share one approach direction, and the approach path must be collision-free.

Skill parameters. Grasps are accepted only if the gripper is collision-free along the insertion path [9]. The grip force is twice the friction-cone bound $(f_{\text{seat}} + mg)/(2\mu_{\min})$. The lateral error bound of an insertion is $\bar{e} = e_{\text{hole}} + e_{\text{tip}} + e_{\text{hand}} + e_{\text{contact}}$, where $e_{\text{contact}} = \mu F/K$ is the deflection of the compliant hand under the pressing force F and Cartesian stiffness K. The spiral search has a pitch below twice the capture width w and ends one turn beyond $\bar{e} - w$ [48]. The simulated fit has a 0.1 mm diametral clearance [49]. Dowel–wood contacts have a stiffness of 2×10^4 N/m, a time constant of 2 ms, and a radial clearance of 0.05 mm.

Robot and perception. All motions use a Cartesian impedance controller with $K_{\text{trans}} = 3000$ N/m and $K_{\text{rot}} = 300$ Nm/rad [50]. The wrist camera's viewing distance keeps all features in view for any pose within the current error bound. A held part is localized at a fixed upward-looking camera. In simulation, *localize* returns the true pose perturbed by noise drawn uniformly within an assumed bound. At close range, the bound is 0.3 mm at the centroid of the observed features and grows with distance from it. At far range, it is 1 mm and 5 mrad.

Retries. Inside a success region, each attempt fails with probability at most $1 - p_{\min} = 0.2$. All $N = 3$ attempts then fail with probability at most $0.2^3 = 0.008$ if attempts are independent. Attempts that reuse the same pose estimate are positively correlated, so this bound is optimistic.

Measuring success regions. We assume that a larger pose-estimate error never makes success more likely. Errors are injected on circles of fixed radius, in a low-discrepancy order of directions. Exploration fits a non-increasing success curve to batches at three radii around the current estimate [51]. Its executions are not reused. Elimination compares grasp and force settings by successive halving [52]. Confirmation tests the radii $\hat{r} - \Delta$, $\hat{r}$, and $\hat{r} + \Delta$ in increasing order, with $n = 40$ new executions each. A test accepts if the one-sided 95% Clopper–Pearson lower bound of the success rate is at least $p_{\min}$. Each test stops once its decision is fixed, and the sequence stops at the first rejection. Testing in this fixed order keeps the overall confidence at 95% [42].

Composition check. Before execution, a composition check tests whether *localize* brings the error inside the success region. It requires $q_{\text{target}} + q_{\text{held}} \le r^\star$, where each q is the largest error that *localize* left over at least 59 simulated runs, for the target and the held part. This maximum exceeds the 95% quantile of the error with 95% confidence [53]. For the first SUNDVIK operation, *localize* leaves at most 0.503 mm at the hole over 60 runs and at most 0.689 mm at the held dowel over 59 runs. The sum, 1.19 mm, passes the check with a margin of 0.31 mm. For part 3, the sum is 0.98 mm.

Table 3: **Skill level and end-to-end assembly** of the SUNDVIK chair frame in MuJoCo, 40 runs. The success region is the largest pose-estimate error at which *insert* succeeds with probability at least $p_{\min} = 0.8$. Failure rates are one-sided 95% Clopper–Pearson upper bounds, and the task-level BT of the end-to-end runs is synthesized with them. Intervals are two-sided 95% Clopper–Pearson intervals.

Success region, dowel	$r^\star = 1.5$ mm: 39/40 at 1.0 mm, 38/40 at 1.5 mm, rejected at 2.0 mm (5/12)
Success region, part 3	$r^\star = 1.5$ mm: 40/40 at 1.5 mm, rejected at 2.0 mm (20/24)
Initial pose-error bound	3.34 mm (dowel), 3.77 mm (part 3)
Error bound after *localize*	1.19 mm (dowel), 0.98 mm (part 3), composition check
Failure rates, dowel	visible $\leq$0.149, undetected $\leq$0.072 (40 executions)
Failure rates, part 3	visible $\leq$0.072, undetected $\leq$0.072 (40 executions)
Task success	35/39 runs [0.76, 0.97], 20/39 without a help request
Redone without help	7 failed attempts, in 6 completed runs
Runs without a result	1 (simulator crash)
Stopped runs	4, a fixed part had to be disassembled
Help requests	16: failed grasps (10), occluded part (2), failed release (2), no contact (2)
Skills	*pick_up* 311/341, *localize* 628/634, *insert* 296/311, all after *localize*
Undetected failures per run	0.25 predicted, 0 observed in 39 runs
Operator interactions per run	1.0 predicted, 1.41 observed

G Parameters and task-level evaluation

Table 4: **Model parameters, part 1 of 2:** costs, operation, and interpretation parameters of the task-level results. Assumed values are design choices or estimates. The end-to-end runs of Q3 instead use the skill failure rates measured in Table 3. Measuring these rates for every product is future work.

Symbol	Parameter	Value	Source / status
$c_{\mathrm{asm}}(h)$	operation cost	$1 + 0.25\, n_h$ (n_h fasteners), in units of one operation without fasteners	assumed
$c_{\mathrm{dis}}(h)$	disassembly cost	$0.5\, c_{\mathrm{asm}}(h)$	assumed
c_s	inspection cost (visual, force)	0.2, 0.3	assumed
c_{ask}	operator confirms or corrects an interpretation	2	assumed
c_{help}	help request (operator completes the operation)	20	assumed
p_{obs}	visible operation failure (retried inside ASSEMBLE)	0.10	assumed, insertion is difficult [54]
N	attempts per operation before a help request	3	design choice
m_h	probability that the decoded interpretation of operation h is wrong	per operation	structured decoding, temperature fitted leave one product out
$\mathcal{R}_h$	prediction set	per operation	split-conformal threshold over products, $\alpha = 0.1$
n_q	interpretations of $\mathcal{R}_h$ shown to the operator	4	design choice

Table 5: **Model parameters, part 2 of 2:** failure and inspection parameters. Triples a/b/c list misinterpretation, connector, and orientation failures.

Symbol	Parameter	Value	Source / status
ε_κ	fault probability per fastener	dowel .03, screw/bolt .02, barrel nut .05, peg .02, direct .01, other .02	assumed
ε_o	orientation fault per attempt	0.02	assumed
β_f	reveal by a later operation	misinterpretation .5, connector .3, orientation .6	assumed
$d_{\text{vis}}(f)$, a_{vis}	visual inspection at the step	misinterpretation .326, connector .113, orientation .202, false alarm .103	measured on 389 rendered line drawings of 30 IKEA-Manual products [55], near chance for connectors
$d_{\text{force}}(f)$, a_{force}	force inspection at the step	.2/.8/.3, false alarm .05	assumed
$\tilde{d}_{\text{vis}}(f)$	visual inspection on a covered (carried) failure	.326/0/.202	as at the step, covered fasteners are not visible
$\tilde{d}_{\text{force}}(f)$	force inspection on a covered failure	.2/.2/.3	assumed
ε_r	late repair leaves an undetected failure	0.05	assumed

Protocol. BTs synthesized on the decoded plans are run in a Monte Carlo simulation of the task model, with the annotated plans as ground truth. In the main comparison, the fault rates $\varepsilon_\kappa, \varepsilon_o$ are 1.5 times and the reveal rates β_f 0.8 times those of the task-level planner. Misinterpreted steps are wrong with probability one. Cost reductions are $1 - \sum \text{synthesized} / \sum \text{baseline}$ over the 21 products, with 95% paired bootstrap intervals (10^4 resamples). At matched risk, the risk bound of the task-level planner is the baseline's planned risk on each product.

Mismatched rates. The 180 settings of Sec. 7 combine fault-rate factors $\{0.5, 1, 1.5, 2, 3\}$, reveal-rate factors $\{0.5, 0.8, 1\}$, cost ratios $c_{\text{ask}}/c_{\text{help}} \in \{0.05, 0.1, 0.25, 0.5\}$, and visual detection rates as measured or shifted by ± 0.2. Simultaneous intervals use the studentized maximum over the 180 settings (Table 6).

Table 6: **Cost reduction (%) of the synthesized BT** at each rule's planned risk over the 180 settings of mismatched rates. Median and maximum are over the 45 settings of each cost ratio. Sig.: number of settings whose 95% lower bound exceeds zero, for simultaneous / pointwise intervals.

	vs. myopic rule + force			vs. KnowNo-style + force			vs. every step / always query
$c_{\text{ask}}/c_{\text{help}}$	median [95% CI]	max	sig.	median [95% CI]	max	sig.	median
0.05	0.1 [0.0, 0.3]	0.1	0/0	0.0 [0.0, 0.0]	0.1	0/0	29.3 / 0.4
0.1	0.3 [0.0, 0.6]	0.3	0/45	0.0 [0.0, 0.0]	0.0	0/0	26.0 / 0.5
0.25	0.1 [0.0, 0.3]	1.0	0/15	0.0 [0.0, 0.0]	0.4	0/0	16.0 / 1.9
0.5	4.9 [1.0, 9.3]	10.2	0/45	0.0 [0.0, 0.0]	3.2	7/15	11.1 / 3.0

Planner's model. We also varied the visual accuracy over $\{0.3, 0.5, 0.7, 0.9\}$, the four cost ratios, and the budget between the least achievable risk and the risk of no inspection. Within the planner's own model, the synthesized tree is then 3.4–22.1% cheaper than the cheapest hand-designed rule that meets the same budget. At matched risk, its gain over the strongest rule is 0.0–1.5%.

Further results. Queries before execution concern the steps with the largest expected cost reduction under the decoder (largest value of information). Of 41 queries in 17 products, 11 changed the decoded tree. They recover six of the nine operations decoded as joining nothing, and they lower the cost of the synthesized BT by 22.5% [4.8, 38.8] (cost 697.9, 4.21 undetected failures, 3 missed operations). The query rules gain as much. Three planners differ only in their error probabilities: one constant (the mean of the decoded m_h), the decoded per-step m_h, and oracle values $m_h \in \{0, 1\}$

from the annotations. Each is evaluated at the risk of the synthesized BT. For the trees of Table 2, which minimize cost $+\,20\times$ risk, the oracle saves 39.8 [20.0, 65.1] cost units over the constant, and the decoded per-step values recover 3% [−11, 22] of this saving at the default query cost and 2% [−92, 43] when a query costs half a help request. For trees that meet the risk bound $\delta = R_{\min} + 0.25\,(R_0 - R_{\min})$, a quarter of the way from the least achievable risk $R_{\min}$ to the risk R_0 of the tree without inspections, they recover 14% [−8, 43] and 59% [8, 85]. A planner that ignores covering sets $\tilde{d}_s(f) = d_s(f)$, as in classical inspection allocation. For the objective of Table 2, it computes a risk of 1.82 for trees whose risk in the planner's model is 2.75. Under the bound δ above, or the tighter one with 0.1 in place of 0.25, its trees exceed δ on 7 of the 21 products. The number n of interpretations shown can also be a choice of the dynamic program, with a query cost proportional to $\log_2(n+1)$ [56]. Under the risk bound δ above, showing the whole prediction set then costs 24.6% [11.4, 38.0] less than showing four. Choosing n per operation adds no further gain (−2.7% [−9.5, 0.2] against the whole set). With SequenceWithMemory, a tree can report a disassembled operation as done. Depending on how an inspection of a disassembled operation is modeled, this raises the risk by 39% or the cost by 11%. With ReactiveSequence and asynchronous skills, 95% of the episodes do not terminate. On the 21 products, a randomized mixture of two trees lowers the cost by 0.1–1.3% at the same risk.